

1. Finding and Exploiting the Vulnerability (Broken Session Management)

Once logged in, look for the challenge titled **"What is Broken Authentication and Session Management?"**. This matches our first required report entry.

1. Click on the lesson to open it. You will see a button that says **"Complete This Lesson"**. If you click it right now, it won't let you pass because the app tracks completion via a client-side cookie.
2. Press **F12** (or right-click anywhere on the page and select **Inspect**) to open the Firefox Developer Tools.
3. Click on the **Storage** tab at the top of the developer panel, expand the **Cookies** dropdown on the left, and click on the target URL.
4. Look for a cookie named **lessonComplete**. Right now, its value is set to **lessonNotComplete**.
5. Double-click the value **lessonNotComplete** and manually alter it to say exactly:

lessonComplete

6. Close the developer panel and click the **"Complete This Lesson"** button on the webpage.
7. The page will refresh and display a long, unique text string called a **Result Key** (e.g., **8781407654...**).
8. Copy that result key, paste it into the **"Submit Result Key Here"** box at the top of the page, and hit **Submit**.

2. What is Cross Site Scripting (XSS)?

Step-by-Step Exploitation

1. Go to the challenge page in your browser where the search box is visible.
2. Copy any of the standard payload vectors below and paste it directly into the **"Please enter the Search Term that you want to look up"** input field:
Option A (Standard Script Tag):
3. `<script>alert('XSS')</script>`

OR

``

4. Click the **Search** or **Submit** button.

5. The page will reload, and an alert pop-up window reading **"XSS"** will instantly flash on your screen, confirming that your script successfully executed inside the browser context!

3. Cross site scripting one

The developer block text gives away the constraint immediately: *"It would appear that your input is been filtered!"* In this specific Security Shepherd lesson, the application backend includes a basic filter that scans your input for classic `<script>` tags. If it detects a script tag, it strips it out or blocks the rendering to thwart basic injection attempts.

However, we can bypass this constraint completely by avoiding the `<script>` tag entirely and using an HTML event handler attribute tied to an alternative tag (like a broken image asset) to force execution.

Step-by-Step Bypass Exploitation

1. Navigate to the Cross Site Scripting One challenge form page in your browser.
2. Clear out the search field input block.
3. Copy and paste the following broken image event handler string exactly into the input line:

```

```

4. Click the Search or Submit button.

Why this structure successfully executes:

- The Filter Bypass: The backend character block is specifically searching for strings containing `<script>` or `</script>`.